# 15/9 Ingeniería de Software en Contexto y Conceptos Fundamentales. (Debrief)

## 1) ¿Qué es Software?

- **Definición práctica:** conjunto de **programas** + **documentación** + **artefactos** que permiten **definir, desarrollar, operar y mantener** el producto (manuales, diagramas, configuraciones, scripts, etc.).
- **Tres grandes familias:**
  - **Software de sistema** (SO, drivers, middleware).
  - **Utilitarios** (herramientas de apoyo, mantenimiento, monitoreo).
  - **Software de aplicación** (ERP, apps web/móvil, sistemas de gestión, etc.).
- **Software = conocimiento empaquetado** en distintos niveles de abstracción (desde requerimientos y arquitectura hasta código ejecutable).

> Ejemplo corto: una app de "Gastos Familiares" incluye el código, la DB y también las US/criterios de aceptación, políticas de backup, manuales y scripts de despliegue.

### Importancia de la documentación en ambos enfoques.

Aunque a veces subestimada, la documentación es un componente intrínseco del producto de software. Es fundamental para la **evolución del producto**, ya que permite que futuros desarrolladores comprendan las decisiones y definiciones iniciales.

En **metodologías ágiles**, se enfatiza la entrega de **software operativo sobre la documentación exhaustiva**, pero esto no significa la ausencia de documentación. Más bien, se busca **generar documentación cuándo sea necesario** y cuando agregue valor al producto y esté centrada en el cliente.

🔥 **Comparación (Ágil vs. Tradicional).**
Mientras que los **procesos dirigidos por un plan** (como el modelo en cascada) son conducidos por una **documentación exhaustiva**, los **métodos ágiles** buscan **simplificar la burocracia** al evitar el trabajo con valor dudoso a largo plazo y eliminar documentación que quizás nunca se use. Sin embargo, un código de alta calidad y legible es clave para el mantenimiento, con o sin documentación formal.

---

## 2) 5 razones para **no** comparar software y manufactura

1. **Menor predictibilidad:** los resultados dependen de decisiones intelectuales cambiantes.
2. **No hay producción en masa:** cada solución es única (contexto, stack, restricciones).
3. **No todas las fallas son "errores" de programación:** muchas provienen de **supuestos** o **requisitos**.
4. **El software no se "gasta"** físicamente; **envejece** por cambios en el entorno o por **mala evolución**.
5. **No está gobernado por leyes físicas** en la misma forma que los bienes manufacturados; predomina el **trabajo intelectual** y la coordinación humana.

---

## 3) Breve historia y motivación

El término "ingeniería de software" surgió en 1969 en una conferencia de la OTAN, en respuesta a los **problemas crecientes en el desarrollo de grandes sistemas de software**: retrasos, falta de funcionalidad, sobrecostos y falta de fiabilidad.

Desde entonces, el progreso ha sido notable, y la sociedad moderna depende críticamente de sistemas de software complejos y fiables.

- 1968: nace el término **Ingeniería de Software** (Conferencia de la OTAN).
- 1975–1995: Brooks (Mythical Man-Month, No Silver Bullet); Humphrey (SP/CMM); auge OO.
- 2001: **Manifiesto Ágil**; 2003: **Lean Software Development**; consolidación de prácticas modernas.
- Motivadores: **retrasos**, **sobrecostos**, **baja calidad** y **requisitos cambiantes** en sistemas cada vez más críticos.

---

## 4) Problemas frecuentes, éxitos y fracasos

### Clasificación de Errores: Las 4 P's

Los errores clásicos en proyectos de software se clasifican en cuatro categorías principales:

**Gente (People):** Problemas relacionados con el personal del proyecto, sus habilidades, comunicación y motivación.

**Errores de la Gente:**

- **Falta de capacitación y experiencia:** La "ignorancia" puede ser más costosa que la capacitación en una actividad humano-intensiva como el software. La selección y desarrollo del personal son cruciales.
- **Problemas de comunicación:** La falta de comunicación eficaz con los participantes puede llevar a construir "una solución elegante para el problema equivocado".
- **Falta de motivación o compromiso:** Un equipo sin un enfoque común en la meta de entregar software funcional puede fracasar. Los ingenieros de software motivados por el trabajo pueden desviarse para "arreglar" cosas innecesarias o ignorar estándares.
- **Mitos del desarrollo:** Creer que un manual de estándares es suficiente o que añadir más programadores a un proyecto atrasado resolverá el problema (el "concepto de la horda de mongoles") son mitos peligrosos.

---

**Proceso (Process):** Ineficacias en las metodologías, procedimientos y flujo de trabajo.

**Errores de Proceso:**

- **Requerimientos mal definidos o inestables:** Si los requerimientos cambian constantemente y no se gestionan (administración de requerimientos), el proyecto puede desviarse.
- **Falta de un plan sólido:** No tener un "mapa" claro del proyecto (plan del proyecto de software) es un riesgo significativo.
- **Proceso inflexible o burocrático:** Modelos de proceso prescriptivos aplicados dogmáticamente pueden aumentar la burocracia y las dificultades. La documentación excesiva sin valor real también es un problema.
- **Falta de SQA (Aseguramiento de Calidad del Software):** No establecer una infraestructura de apoyo para la calidad o no realizar auditorías y revisiones técnicas.

---

**Producto (Product):** Deficiencias en la calidad, funcionalidad o requisitos del software final.

**Errores de Producto:**

- **Falta de confiabilidad o funcionalidad:** El software debe ser de alta calidad; si falla, las consecuencias pueden ser catastróficas. La calidad debe inyectarse a lo largo del proyecto, no solo buscarse al final.
- **Dificultad de mantenimiento:** Si el software no está bien diseñado y documentado, es difícil de corregir, adaptar o aumentar, lo que aumenta los costos a largo plazo.
- **Problemas de usabilidad o rendimiento:** Un software que no es fiable o es demasiado lento será rechazado por los usuarios.
- **Tecnología (Technology):** Desafíos o elecciones inadecuadas de herramientas, plataformas y técnicas.

**Problemas típicos:** requerimientos incompletos o cambiantes, expectativas poco realistas, falta de involucramiento del usuario, rotación/skills, documentación deficiente.

- **Cuando nos va bien:** usuario involucrado, apoyo gerencial, requisitos claros, planificación realista, hitos intermedios, equipo competente.
- **Conclusión pedagógica:** *"Saber programar **no** es suficiente"*. La disciplina integra **personas + proceso + producto + tecnología**.

> El **vuelo 501 del cohete Ariane 5** es un ejemplo catastrófico de un proyecto de software fallido debido a **errores de especificación y diseño del sistema de referencia inercial**. Este evento subraya la importancia crítica de la **precisión en la especificación y la robustez en el diseño** en sistemas donde las fallas tienen consecuencias severas, como la pérdida de equipo y, potencialmente, vidas humanas. El estudio de este caso es tan relevante que Sommerville lo incluye como material adicional en su libro.

---

## 5) Ingeniería de Software (IS): la disciplina

- **Definición:** aplicación **sistémica** de principios de ingeniería a **todos los aspectos** del ciclo de vida del software (desde especificación hasta mantenimiento), bajo **restricciones** de tiempo, costo y calidad.

*La Ingeniería de Software es una **disciplina de ingeniería que se interesa por todos los aspectos de la producción de software**, abarcando desde las etapas iniciales de especificación hasta el mantenimiento posterior a la operación.*

- Esto implica que los ingenieros de software, al igual que otros ingenieros, **aplican teorías, métodos y herramientas** de manera selectiva, buscando soluciones a problemas incluso cuando no existen teorías o métodos preexistentes. También deben operar bajo **restricciones organizacionales y financieras**.
- **Ética profesional:** prioridad a la **seguridad** y **bienestar** del público; conducta responsable (ACM/IEEE Code).

**<u>Fundamentos de la Ingeniería de Software:</u>**

- Los pilares de la Ingeniería de Software incluyen el **proceso, la confiabilidad, los requerimientos, la gestión y la reutilización**. Estos conceptos son inherentes a todo el desarrollo de software profesional.

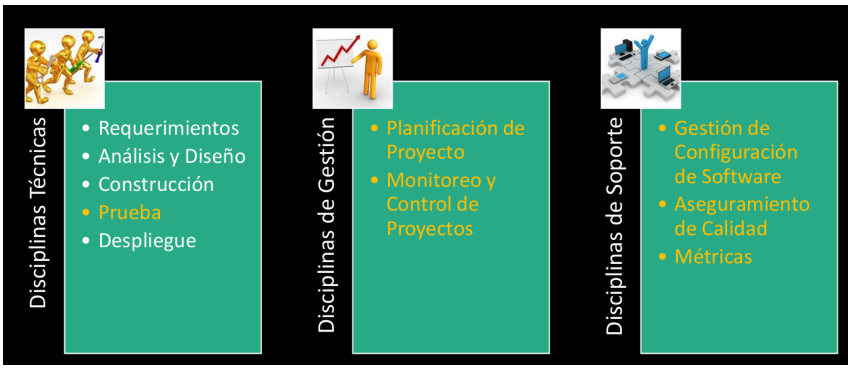
Las `cinco actividades estructurales generales` de la ingeniería de software son:

- **Comunicación:** Interacción con el cliente y otros participantes para entender objetivos y reunir requerimientos.
- **Planeación:** Creación de un "mapa" del proyecto, definiendo tareas, riesgos, recursos y una programación.

- **Modelado:** Creación de "bosquejos" o representaciones del software para comprender mejor el problema y su solución. Esto incluye análisis y diseño.
- **Construcción:** Generación de código y pruebas.
- **Despliegue:** Entrega del software y obtención de retroalimentación.

Además, existen **actividades sombrilla** que se aplican a lo largo de todo el proceso, como el aseguramiento de la calidad del software (SQA), revisiones técnicas, medición, administración del cambio y administración del riesgo.

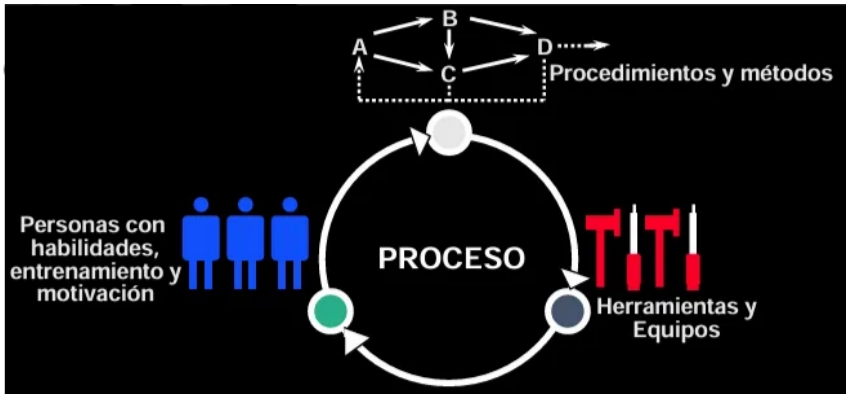
### 5.1) La materia en contexto: disciplinas




- **Técnicas:** Requerimientos, Análisis/Diseño, Construcción, **Pruebas**, Despliegue.
- **Gestión:** Planificación, Monitoreo y Control del proyecto.
- **Soporte:** Gestión de Configuración (SCM), **Aseguramiento de Calidad**, **Métricas**.

---

## 6) Proceso de Software




DEFINICIONES.

***Proceso***: La secuencia de pasos ejecutados para un propósito dado (IEEE)
***Proceso de Software***: Un conjunto de actividades, métodos, prácticas, y transformaciones que la gente usa para desarrollar  o mantener software y sus productos asociados (Sw-CMM)

Definición **FINAL:** conjunto **estructurado** de actividades, métodos, prácticas y transformaciones para **desarrollar** y **mantener** software y sus artefactos.

Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse.
▪ Debe ser explícitamente modelado si va a ser administrado.

- **Actividades genéricas:**
  - **Comunicación** (elicitación)
  - 2) **Planificación**
  - 3) **Modelado** (análisis y diseño)
  - 4) **Construcción** (código y pruebas)
  - 5) **Despliegue** y feedback.
- **Actividades sombrilla:** SQA, revisiones técnicas, medición, gestión de cambios y de riesgos, gestión de configuración.

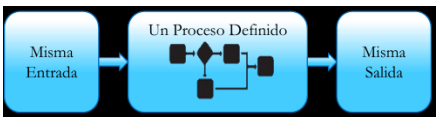
## Modelos de proceso más citados

- **Cascada (secuencial):** fases en serie; visibilidad documental; útil con requisitos bien comprendidos.
- **Incremental/iterativo:** entrega por **incrementos**; refina requisitos/diseño con feedback; útil con cambio.
- **Orientado a reutilización:** integra componentes existentes (COTS/OSS) para reducir costo/tiempo.
- **Espiral:** gestión de **riesgos** explícita por ciclos (objetivos → alternativas → análisis de riesgo → mitigación).
- **RUP:** buenas prácticas organizadas por **fases** (concepción, elaboración, construcción, transición).
- **Ágil (Scrum, XP):** entrega rápida e incremental; foco en colaboración y valor temprano.

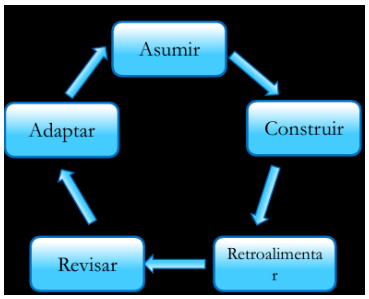
## 6.1) Proceso **definido** vs **empírico** IMPORTANTE!

- **Definido (plan-driven):** asume **repetibilidad**: misma entrada → misma salida. Útil con **requisitos estables** y contextos **regulatorios**. Riesgo: rigidez ante el cambio.

*(inspirados en las líneas de producción)*




- **Empírico (ágil):** asume **variabilidad**; gobierna con **inspección** y **adaptación** (ciclos cortos, reviews, retros). Requiere cliente involucrado y disciplina de equipo.




Asume procesos complicados  con variables cambiantes. Cuando se repite el proceso, se pueden llegar a obtener resultados diferentes.
La administración y control es a través de inspecciones frecuentes y adaptaciones. Son procesos que trabajan bien con procesos creativos y  complejos.

- **Analogía didáctica:** *"caminos en el pasto"* (UCI): observamos por dónde **realmente** camina la gente y **adaptamos** las sendas.

> [Contenido propio] Heurística: si el costo del cambio es alto → más definido; si el costo del conocimiento (incertidumbre) es alto → más empírico.

---

## 7A) Proyecto de software.

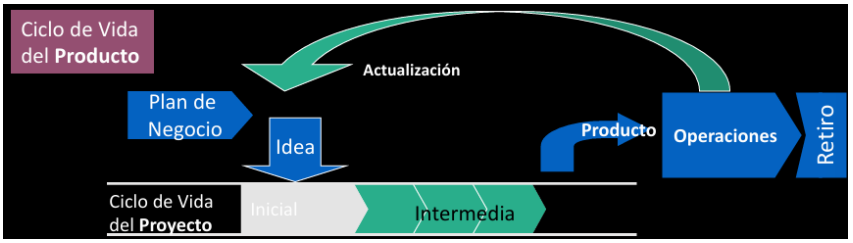
**Definición y Características Clave:**

- Un **proyecto de software** es un **esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único**. Su naturaleza temporal lo diferencia de las operaciones continuas.
- Sus **características** incluyen:
  - **Únicos:** Cada proyecto tiene particularidades que lo hacen único, incluso si son similares a otros (ej. cronogramas diferentes).
  - **Orientado a objetivos:** Los objetivos deben ser **claros y no ambiguos** para guiar eficazmente el proyecto.

La **administración de proyectos** es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para cumplir sus objetivos. Implica "tener el trabajo hecho" a tiempo, con el presupuesto acordado y habiendo satisfecho las especificaciones.

## 7B) Ciclos de vida

Un ciclo de vida de un proyecto software es un representación de un proceso. Grafica una descripción del proceso desde una perspectiva particular




### Clasificación de ciclos.

- **Secuencial**
- **Iterativo/Incremental**
- **Recursivo**

  *Descripción*: el patrón **requisito→diseño→build→prueba** se **replica** en **múltiples niveles** de descomposición (subsistemas/microservicios), permitiendo mini-ciclos locales que se integran en ciclos macro. **Diferencia** con iterativo: lo **iterativo** agrega valor en el **tiempo**; lo **recursivo** replica el **mismo proceso** a diferentes **niveles estructurales** en paralelo.

> Idea clave: tanto producto como proyecto tienen ciclos de vida; el ciclo del proyecto envuelve al del producto y lo habilita.

---

## 8) Estándares, SQA y V&V (conexión con Calidad)

- **Estándares:** de **producto** (documentos, codificación, documentación) y de **proceso** (pasos, prácticas, artefactos). Beneficios: continuidad, marco de calidad, certificación (ISO 9001, CMMI), buenas prácticas.
- **SQA (Aseguramiento de Calidad):** infraestructura para que métodos/procesos se apliquen y se **verifique** su uso (planes, auditorías, métricas, revisiones técnicas).
- **Independencia de QA:** idealmente separada de desarrollo para objetividad.
- **V&V:**

- **Verificación:** “¿Construimos **correctamente** el producto?” (alineación a especificaciones/estándares; análisis estático, inspecciones).
- **Validación:** “¿Construimos el **producto correcto**?” (satisface necesidades; pruebas de aceptación/sistema).

> [Contenido propio] Conexión con TPs: trazá CA→PA por historia; revisiones por pares antes de merge; política de ramas con chequeos automáticos (lint/tests).

---

## 9) Métricas y medición (visión práctica)

- **Propósitos: señalar** (control), **predecir** (estimación), **evaluar** (costos), **mejorar** (procesos/producto).
- **Tipos:**
  - **De proceso/proyecto:** esfuerzo, tiempo para reparar defectos, burn-down, lead/cycle time.
  - **De producto:** tamaño (SLOC, PF), complejidad ciclomática, fallas, rendimiento.
- **GQM (Goal-Question-Metric):** parte de **objetivos** y deriva preguntas y métricas; evita medir “por medir”.

> [Contenido propio] Consejo: elegí 3–5 indicadores estables por TP (p.ej., cobertura de pruebas, defectos abiertos al cierre, velocidad promedio) y mantenelos iteración a iteración.

---

## 10) Gestión (resumen enlazado a proceso)

- **Plan de proyecto:** organización, WBS, cronograma (Gantt), riesgos, recursos, mecanismos de seguimiento; **evoluciona** durante el desarrollo.
- **Triple restricción:** alcance–tiempo–costo con **calidad** como resultado sistémico.
- **Riesgo:** proyecto (calendario/recursos), producto (calidad/rendimiento), negocio (mercado/estrategia). Gestión: identificación → análisis → monitoreo → mitigación/contingencias.
- **Equipo:** tamaño razonable, balance de skills, cohesión, motivación; liderazgo que facilita y remueve impedimentos.

> [Contenido propio] Antipatrones comunes: “más gente a un proyecto atrasado” (Brooks), documentación sin uso, deuda técnica por falta de refactorización.

---

## 11) Relación Producto vs. Proyecto y medición del alcance

- **Alcance del producto:** capacidades/atributos entregados (incluye artefactos de soporte). Se define en **requerimientos**.
- **Alcance del proyecto:** trabajo necesario para entregar esas capacidades (todas las actividades del proceso).
- **Medición del alcance:** proxies como **tamaño** (SLOC, Puntos de Función), **entregables** cumplidos, y **criterios de aceptación** satisfechos.